Claude Certified Architect – Foundations

Question Bank (Set 1) · 60 questions grouped by exam domain, with answers and explanations

Exam detail	Value (as published by prep sources, Oct 2026)
Exam	Claude Certified Architect – Foundations (CCA-F)
Format	60 scenario-based multiple-choice questions, 120 minutes (2 min per question). Each question has one correct answer and three distractors.
Scoring	Scaled 100–1,000; passing score 720
Delivery	Pearson VUE; closed book

#	Domain	Weight	Questions here
1	Agentic Architecture & Orchestration	27%	16
2	Tool Design & MCP Integration	18%	11
3	Claude Code Configuration & Workflows	20%	12
4	Prompt Engineering & Structured Output	20%	12
5	Context Management & Reliability	15%	9
	Total	**100%**	**60**

Important: These are original practice questions written to match the published CCA-F domain blueprint and the exam's scenario-based style. They are **not** actual exam questions, and Anthropic has not reviewed or endorsed them. Domain weights and format come from third-party prep sources; check them against Anthropic's official exam guide. This set does not share questions with Set 2 (the scenario mock exam).

How to use this bank

1. Answer before you look. Each domain lists its questions first, followed by the answers and explanations.

2. Study the distractors. Every explanation says why each wrong option fails. The exam is built from plausible options that do not fix the actual problem.

3. Pace yourself. Allow 2 minutes per question. All 60 questions take about 2 hours, the same as the real exam.

4. Track by domain. Score each domain with the key at the end. Below 75% in a domain is where to focus.

Principles that decide many CCA-F questions

• **Deterministic beats probabilistic** when a rule must always hold: use hooks, programmatic prerequisites, tool_choice, and schemas rather than stronger prompt wording.

• **Use the API's real signals:** stop_reason ("tool_use" or "end_turn") controls the loop, not parsing text or arbitrary caps.

• **Context is explicit:** subagents do not inherit the coordinator's history, so pass what they need, with metadata.

• **Fix the root cause:** unclear tool descriptions, too-narrow decomposition, and vague criteria are fixed at the source.

• **Errors carry meaning:** separate transient, validation, business, and permission errors, and never report a failure as an empty success.

• **Preserve facts and provenance:** keep case facts outside summaries, keep claim-source mappings, and annotate conflicts.

• **Scope things correctly:** project vs user CLAUDE.md, .mcp.json vs ~/.claude.json, .claude/commands vs ~/.claude/commands.

Domain 1: Agentic Architecture & Orchestration (27%)

Covers: Agentic loops and stop_reason; coordinator-subagent (hub-and-spoke) design; the Task tool and passing context; parallel subagents; hooks for deterministic enforcement; escalation and handoff; task decomposition; session forking and resuming.

Question 1. A developer's agentic loop sends a request, executes any tools Claude asks for, and stops when the response text contains the word "DONE". In testing, the loop sometimes stops after one tool call, and sometimes keeps running after Claude has clearly finished. What is the correct way to control loop termination?

- **A.** Keep the "DONE" check but add a system prompt rule that Claude must always end its final answer with exactly the word DONE on its own line.
- **B.** Stop the loop as soon as a response contains any assistant text block, since text means Claude is addressing the user rather than working.
- **C.** Continue looping while stop_reason is "tool_use" (execute the tools and append the results), and exit when stop_reason is "end_turn".
- **D.** Run a fixed number of iterations sized to the most complex task observed in testing, then return whatever the final response contains.

Question 2. After executing a tool requested by Claude, a developer appends the tool's output to the conversation as a new assistant message containing plain text. Claude then behaves as though the tool was never run and requests it again. What is the fix?

- **A.** Keep the assistant message but prefix the output with "TOOL RESULT:" so Claude can recognise it as the result of the earlier call.
- **B.** Start a fresh request containing only the tool output and the original question, so the earlier tool_use block cannot confuse Claude.
- **C.** Append the output as a system prompt update, so Claude treats the tool's result as authoritative context for the rest of the session.
- **D.** Return the output in a user message as a tool_result content block whose tool_use_id matches the tool_use block Claude produced.

Question 3. An agent's loop uses max_iterations = 5 as its main stopping condition. Complex customer cases are frequently cut off mid-investigation, while simple cases finish in two iterations. What is the best redesign?

- **A.** Use stop_reason as the primary termination signal, and keep a much higher iteration cap only as a safety guardrail with graceful handling.
- **B.** Ask Claude at every iteration whether it believes it is finished, and stop the loop as soon as it answers yes in its text output.
- **C.** Classify each case as simple or complex up front, and assign it a matching fixed iteration budget of 2 or 10 iterations accordingly.
- **D.** Raise max_iterations to 25 so that complex cases have room to finish, while keeping it as the loop's main stopping condition.

Question 4. In a research system, a coordinator first runs a web-search subagent and then spawns an analysis subagent with the prompt "Analyse the findings and identify the key trends." The analysis subagent reports that it has no findings to analyse. What is the root cause?

- **A.** Subagents do not inherit the coordinator's conversation history, so the search findings must be passed explicitly in the subagent's prompt.
- **B.** The two subagents ran in parallel rather than sequentially, so the search results had not yet been written when analysis started.
- **C.** The coordinator's model is too small to share context reliably between subagents, and should be upgraded to a more capable model.
- **D.** The analysis subagent's allowed tools do not include web search, so it could not retrieve the findings that the search subagent already found.

Question 5. A coordinator agent built with the Claude Agent SDK has three AgentDefinitions configured for subagents, but in logs it never delegates. It tries to do every task itself. What is the most likely configuration problem?

A. Subagents must be registered as MCP servers in .mcp.json before the coordinator can discover and invoke them at runtime.

B. The subagents' system prompts are too short, so the coordinator does not consider them capable enough to handle delegated work.

C. The coordinator needs tool_choice set to "any" on every request, otherwise it is not permitted to call other agents.

D. The coordinator's allowedTools does not include the Task tool, which is the mechanism the coordinator uses to spawn subagents.

Question 6. A coordinator delegates four independent literature searches. Logs show it spawns one subagent, waits for its result, then spawns the next, so the total latency is the sum of all four. How should this be fixed?

A. Give one subagent all four search topics in a single prompt, so it can work through them back to back within one context window.

B. Move the four searches to the Message Batches API, so they are processed together asynchronously at a lower cost.

C. Have the coordinator emit all four Task tool calls in a single response, so the subagents run in parallel.

D. Lower max_tokens on each subagent so that every individual search returns faster and the sequential chain finishes sooner overall.

Question 7. A research system is asked about "the impact of AI on creative industries". Each subagent completes its task well, but the final report covers only visual arts and omits music, writing, and film. Logs show the coordinator created three subtasks: "AI in digital art", "AI in graphic design", and "AI in photography". What is the root cause?

A. The document-analysis subagent filtered out sources that it judged insufficiently authoritative for the report.

B. The web-search subagent's queries were too specific and missed sources about music, writing, and film during retrieval.

C. The synthesis agent dropped the non-visual findings because they did not fit the structure of the report it was generating.

D. The coordinator's task decomposition was too narrow, so the subtasks never covered the other creative domains in the question.

Question 8. In a multi-agent system, subagents call one another directly to share intermediate results. The team cannot trace why results are inconsistent, and one failed subagent silently corrupted two others. Which architectural change is recommended?

A. Merge all of the subagents into a single agent with every tool, so there is no inter-agent communication left to trace or debug.

B. Adopt hub-and-spoke: route all communication between subagents through the coordinator, which manages context, errors, and observability.

C. Have each subagent write its results into a shared conversation history that all the other subagents can read and update directly.

D. Add more detailed logging to each subagent while keeping the direct calls, so inconsistencies can be reconstructed after the fact.

Question 9. A support agent's system prompt says "Always verify the customer's identity with get_customer before processing any refund." Audits show the agent skips verification in about 8% of refund cases. Refunds to unverified accounts are a compliance violation. What is the most effective fix?

A. Add a programmatic prerequisite that blocks process_refund unless get_customer has returned a verified customer ID in this session.

B. Add a post-hoc audit job that flags refunds without prior verification, so the support team can reverse them within 24 hours.

C. Strengthen the system prompt with capital letters and repeat the verification rule in both the opening and closing sections of the prompt.

D. Add few-shot examples showing the agent verifying identity before every refund, covering several different types of refund request.

Question 10. Company policy requires that refunds above $500 be handled by a human. The agent sometimes processes $600 to $900 refunds itself. Using the Agent SDK, which approach best enforces this?

A. A hook that intercepts process_refund calls, blocks those over $500, and redirects the case to the human escalation workflow.

B. A PostToolUse hook that logs every refund above $500 after it completes and notifies a supervisor to review the transaction.

C. A system prompt section that explains the $500 limit in detail, including the business reasons for it and examples of compliant behaviour.

D. Remove process_refund from the agent's tools entirely, so a human handles every refund whatever its amount.

Question 11. An agent uses three MCP tools that return dates in different formats: Unix timestamps, ISO-8601 strings, and "MM/DD" without a year. The agent occasionally misorders events and miscalculates warranty periods. What is the most reliable fix?

A. Few-shot examples showing correct reasoning over mixed date formats, including the ambiguous case of dates with no year.

B. A PostToolUse hook that normalises all date fields to one consistent format before the results reach the model.

C. Ask the agent to restate every date in ISO format in its reasoning before using it in any calculation.

D. A system prompt instruction that lists all three date formats and explains in detail how to convert each one before comparing dates.

Question 12. Partway through a straightforward billing question that the agent can resolve, a customer writes: "I'd rather just speak to a human, please." What should the agent do?

A. Escalate to a human promptly, honouring the explicit request, and pass along a summary of the case so far.

B. Run a sentiment check, and escalate only if the customer's frustration score exceeds the configured escalation threshold.

C. Explain that the issue is simple, then resolve it quickly, since resolving it is faster for the customer than waiting in a queue.

D. Ask the customer to describe the issue once more in detail before deciding whether escalation is truly necessary.

Question 13. A support agent escalates many simple password-reset cases but resolves complex policy-exception cases on its own, often incorrectly. Which change best calibrates when it escalates?

A. Add explicit escalation criteria to the system prompt, with few-shot examples contrasting cases to resolve against cases to escalate.

B. Escalate every case that takes more than three tool calls, since longer investigations correlate with greater complexity.

C. Have the agent output a self-rated confidence score from 1 to 10 with each decision, and escalate automatically whenever it is below 7.

D. Add a sentiment-analysis step and escalate whenever a customer's message is classified as negative, because those cases are complex.

Question 14. Human agents receiving escalated cases complain that they must ask customers to repeat everything, because they cannot see the AI conversation transcript. What should the escalation design include?

A. A link to the raw transcript of the conversation, so the human agent can read the whole AI interaction before speaking to the customer.

B. An instruction for the customer to summarise their own issue for the human agent once the transfer is complete.

C. A structured handoff summary: customer ID, issue, root cause found, actions already taken, amounts involved, and the recommended next step.

D. A short free-text note generated by the agent that describes the customer's emotional state and general tone during the conversation.

Question 15. Which task is best handled by dynamic, adaptive decomposition rather than a fixed prompt chain?

A. Reviewing every pull request for the same three aspects (security, style, tests) in the same order, with the same output template.

B. Extracting a fixed set of 12 fields from standard invoices, then validating them, then formatting them into the accounting system's schema.

C. Adding comprehensive tests to a legacy codebase: map the structure, find high-impact areas, then adapt the plan as dependencies appear.

D. Translating product descriptions into five languages, each one going through the same translate-then-proofread sequence every time.

Question 16. After a shared codebase analysis, a developer wants to explore two refactoring strategies independently and compare them, without either exploration contaminating the other. Which session approach fits best?

A. Resume the original session twice, once for each strategy, with --resume so that both continue from the same point in the history.

B. Use fork_session to create two branches from the shared analysis, and explore one strategy in each branch.

C. Continue in the same session, trying the first strategy, then reverting the files and trying the second while the history stays intact.

D. Start two brand-new sessions and paste a short summary of the analysis into each one before exploring the separate strategies.

Answers & Explanations: Domain 1

Questions 1–16

Question 1 · Domain 1: Agentic Architecture & Orchestration

Correct answer: C

The stop_reason field is the API's explicit signal: "tool_use" means Claude is waiting for tool results, and "end_turn" means it has finished. A still parses natural language, which is fragile. B is wrong because a response can contain text and tool_use blocks together. D cuts off complex tasks and wastes calls on simple ones.

Question 2 · Domain 1: Agentic Architecture & Orchestration

Correct answer: D

The Messages API links results to requests through tool_result blocks that carry the matching tool_use_id, sent in the user turn with the full history. C misuses the system prompt. B drops the conversation history Claude needs. A still puts the result in the wrong role and format.

Question 3 · Domain 1: Agentic Architecture & Orchestration

Correct answer: A

Arbitrary caps should not be the primary stopping mechanism. The model signals completion through stop_reason, and caps exist only to stop runaway loops. D keeps the same anti-pattern at a different number. B relies on parsing natural language. C still truncates any case that needs more steps than its budget.

Question 4 · Domain 1: Agentic Architecture & Orchestration

Correct answer: A

Each subagent starts with an isolated context and sees only what is in its own prompt. The coordinator must include the findings it wants analysed. D would make the subagent repeat the search instead of fixing the hand-off. B contradicts the scenario, which ran them in sequence. C confuses model size with how context is scoped.

Question 5 · Domain 1: Agentic Architecture & Orchestration

Correct answer: D

Spawning subagents happens through the Task tool. If the coordinator is not allowed to use it, it cannot delegate, however the agents are defined. B is not what blocks delegation. A confuses subagents with MCP servers. C would force tool calls but does not grant access to the Task tool.

Question 6 · Domain 1: Agentic Architecture & Orchestration

Correct answer: C

Multiple tool calls in one assistant turn allow parallel subagent execution. A serialises the work and mixes contexts. D risks truncated results without removing the serial chain. B has no latency guarantee (up to 24 hours), which is unsuitable for an interactive research flow.

Question 7 · Domain 1: Agentic Architecture & Orchestration

Correct answer: D

The logs show the subtasks themselves excluded the missing domains, so the gap was created at decomposition, before any subagent ran. A, B, and C blame downstream agents that did exactly what they were assigned.

Question 8 · Domain 1: Agentic Architecture & Orchestration

Correct answer: B

Central coordination gives one place for consistent error handling, information routing, and observability. C creates shared mutable state with the same failure propagation. D only adds visibility after the fact. A gives up specialisation and context isolation.

Question 9 · Domain 1: Agentic Architecture & Orchestration

Correct answer: A

When compliance requires a guarantee, enforce the rule in code rather than relying on probabilistic instruction-following. C and D reduce the failure rate but cannot guarantee zero. B detects violations only after the money has moved.

Question 10 · Domain 1: Agentic Architecture & Orchestration

Correct answer: A

Intercepting the tool call before it executes gives a deterministic guarantee for the business rule. C is still probabilistic. B runs after the refund has executed. D over-corrects and gives up automation for the large majority of compliant refunds.

Question 11 · Domain 1: Agentic Architecture & Orchestration

Correct answer: B

Deterministic transformation of tool outputs removes the ambiguity before the model sees it. A, C, and D all leave the conversion to probabilistic model reasoning on every call.

Question 12 · Domain 1: Agentic Architecture & Orchestration

Correct answer: A

An explicit request for a human should be honoured right away. C overrides the customer's stated preference. B uses sentiment as a proxy when the request is explicit. D adds friction and delay.

Question 13 · Domain 1: Agentic Architecture & Orchestration

Correct answer: A

Clear criteria and contrasting examples teach the decision boundary directly. Self-reported confidence (C) is poorly calibrated: the agent is already confidently wrong on the hard cases. D measures emotion, not complexity. B is a crude proxy that misfires both ways.

Question 14 · Domain 1: Agentic Architecture & Orchestration

Correct answer: C

A concise, structured handoff gives the receiving human everything needed to act at once. A is unavailable in this scenario and slow to read. D omits the facts of the case. B pushes the work back onto the customer.

Question 15 · Domain 1: Agentic Architecture & Orchestration

Correct answer: C

Open-ended investigations, where the next steps depend on what is discovered, need adaptive decomposition. A, B, and D have predictable, repeatable structures that suit fixed prompt chains.

Question 16 · Domain 1: Agentic Architecture & Orchestration

Correct answer: B

Forking creates independent branches that share the same baseline context. C mixes both explorations in one history. D loses the detail of the original analysis. A resumes one shared session, so its history is not split into two separate branches.

Domain 2: Tool Design & MCP Integration (18%)

Covers: Tool descriptions and boundaries; structured error responses; scoping tools per agent; tool_choice (auto, any, forced); MCP server scoping (.mcp.json vs ~/.claude.json); environment variable expansion; MCP resources; built-in tools.

Question 17. An agent has two tools: analyze_content ("Analyses content") and analyze_document ("Analyses a document"). One handles web pages and the other handles uploaded PDFs, but the agent picks the wrong one about a quarter of the time. What is the most effective fix?

- **A.** Add a routing classifier in front of the agent that predicts the correct tool and passes it to the model as a forced tool_choice.
- **B.** Merge both into one analyze tool that detects internally whether its input is a web page or a PDF and processes it accordingly.
- **C.** Rename the tools to distinguish their purpose, and expand each description with its input types, example queries, and when to use the other one.
- **D.** Add five few-shot examples to the system prompt showing correct selections, while keeping the existing tool names and descriptions unchanged.

Question 18. A support agent has get_customer ("Retrieves customer information") and lookup_order ("Retrieves order details"). When a customer gives only an order number, such as ORD-48213, the agent often calls get_customer first, which fails. What should be changed?

- **A.** Set tool_choice to force lookup_order on the first turn of every conversation, so order details are always retrieved first.
- **B.** Make get_customer also accept order numbers, so either tool succeeds whichever one the agent happens to choose for the request.
- **C.** Add a system prompt instruction to always call lookup_order before get_customer, whatever information the customer provides.
- **D.** Expand both descriptions with accepted input formats (for example, order IDs like ORD-#####), example queries, and edge cases.

Question 19. When anything goes wrong, a tool returns {"error": "Operation failed"}. The agent cannot tell whether to retry, ask the customer for corrected input, or explain a policy limit. How should the error response be designed?

- **A.** Return structured metadata: an error category (transient, validation, business, permission), an isRetryable flag, and a readable description.
- **B.** Return the same generic message but add a system prompt rule telling the agent to retry every failed tool call exactly three times.
- **C.** Return the HTTP status code and the raw stack trace, so the agent has all the low-level technical information needed for diagnosis.
- **D.** Suppress errors entirely and return an empty result, so the agent can continue the conversation without getting stuck on failures.

Question 20. When the order database times out, the order-search tool returns an empty list. The agent then tells customers "You have no orders with us", which upsets them. What is the correct tool design?

- **A.** Cache the last successful result for each customer and return the cached orders whenever a live query fails or times out.
- **B.** Add a system prompt instruction that an empty result may mean a timeout, so the agent should apologise and ask the customer to retry.
- **C.** Increase the database timeout so that searches complete more often before the tool needs to return any response at all.
- **D.** Distinguish failures from valid empty results: return an error flagged as a timeout and retryable, rather than an empty success result.

Question 21. A refund tool rejects a request because it exceeds the 30-day policy window. The agent repeatedly retries the call. Which error response best supports correct agent behaviour?

A. A transient error with a suggested retry delay, so the agent waits briefly before trying again later in the same conversation.

B. A permission error indicating that the agent lacks rights, so the agent asks the user to log in again with different credentials.

C. An error categorised as a business rule violation, with isRetryable set to false and a customer-friendly explanation of the policy.

D. A generic failure message with an internal error code that the support team can look up in the documentation if needed.

Question 22. In a research system, the synthesis subagent has access to all 18 tools, including web search and document fetching. It frequently starts new searches instead of synthesising, and its tool selection is erratic. What is the best fix?

A. Keep all 18 tools available but lengthen each description, so the agent better understands which tools apply to synthesis.

B. Add a system prompt instruction telling the synthesis agent that it must not use search tools unless absolutely necessary.

C. Set tool_choice to "none" for the synthesis agent, so it cannot call any tools during its work in the research pipeline.

D. Restrict each subagent to the small set of tools relevant to its role, so the synthesis agent no longer has the search tools.

Question 23. A metadata-extraction pipeline must always call extract_metadata first. Enrichment tools may only be chosen afterwards. Sometimes Claude calls an enrichment tool first. How should this be enforced?

A. Set tool_choice to "any" on the first request, so that Claude is required to call some tool before it produces text.

B. List extract_metadata first in the tools array, since Claude prefers tools that appear earlier in the definition list.

C. Set tool_choice to force extract_metadata on the first request, then use "auto" for the follow-up turns.

D. Add a system prompt rule stating that extract_metadata must always be called first, before any enrichment tools are considered.

Question 24. An extraction service receives documents that may be invoices, receipts, or contracts, each with its own extraction tool. Sometimes Claude replies in plain text instead of calling any tool, which breaks the pipeline. The document type is not known in advance. What is the best setting?

A. Merge the three tools into one schema whose fields are the union of all three, and force that single tool on every request.

B. Use tool_choice "any", so Claude must call one of the extraction tools while still choosing which one fits the document.

C. Use tool_choice "auto" with a stronger instruction to always call a tool, while keeping the three separate extraction tools.

D. Force the invoice tool for every document, since invoices are the most common type in this pipeline's incoming traffic.

Question 25. A team wants every developer's Claude Code to connect to the GitHub MCP server, but no one may commit a personal access token to the repository. What is the correct configuration?

A. Commit .mcp.json with a shared service-account token, since the repository is private and only team members can read it.

B. Store the token in CLAUDE.md, so Claude Code can read it and pass it to the MCP server when it starts each session.

C. Ask each developer to add the server to their user-level ~/.claude.json, following setup steps documented in the README.

D. Commit a project-scoped .mcp.json that references the token through environment variable expansion, such as ${GITHUB_TOKEN}.

Question 26. A generic analyze_document tool handles extraction, summarisation, and claim verification, chosen by a mode parameter. The agent often uses the wrong mode, and outputs vary unpredictably. What is the recommended redesign?

A. Add a validation step that rejects calls whose mode appears inconsistent with the request, and asks the agent to retry with another mode.

B. Keep the single tool and add an enum with detailed descriptions for each value of the mode parameter, plus examples of when to use each.

C. Split it into purpose-specific tools (extract_data_points, summarize_content, verify_claim_against_source) with defined inputs and outputs.

D. Remove the mode parameter, and have the tool run all three operations on every call and return the combined output to the agent.

Question 27. An agent working with an internal documentation MCP server makes many exploratory tool calls (list folders, open pages, list again) just to learn what documentation exists before it can answer anything. How can the MCP server reduce this overhead?

A. Increase the agent's tool-call limit, so exploration can complete fully before the agent has to start answering.

B. Expose a catalogue of the available documents as MCP resources, so the agent can see what exists without exploratory tool calls.

C. Add a tool that returns the full text of every document in one call, so the agent never needs to explore folders.

D. Add a system prompt listing every document title, and update the prompt by hand whenever documentation changes.

Answers & Explanations: Domain 2

Questions 17–27

Question 17 · Domain 2: Tool Design & MCP Integration

Correct answer: C

Tool descriptions are the main input to tool selection, so fix the ambiguity at the source with clear names and boundaries. A adds infrastructure to work around a documentation problem. D adds tokens and leaves the confusing definitions in place. B can work, but it hides distinct behaviours behind a vague interface instead of making them clear.

Question 18 · Domain 2: Tool Design & MCP Integration

Correct answer: D

Minimal descriptions give the model no basis for choosing. Documenting input formats and examples lets it map an order ID to the right tool. C creates a rigid rule that is wrong when the customer gives a name. B blurs tool responsibilities. A forces a call that is wrong when no order is involved.

Question 19 · Domain 2: Tool Design & MCP Integration

Correct answer: A

Structured, categorised errors let the agent choose the right recovery: retry, ask for input, or explain the limit. C is noisy and not actionable. B retries non-retryable errors. D hides failures and causes wrong answers.

Question 20 · Domain 2: Tool Design & MCP Integration

Correct answer: D

An access failure reported as "no results" is a silent error that misleads the agent. C reduces how often it happens but keeps the ambiguity. B forces the agent to guess on every empty result. A can serve stale or wrong data.

Question 21 · Domain 2: Tool Design & MCP Integration

Correct answer: C

The agent needs to know this is a non-retryable business rule, plus wording it can relay or escalate on. A and B misclassify the error and drive the wrong recovery. D gives the agent nothing to act on.

Question 22 · Domain 2: Tool Design & MCP Integration

Correct answer: D

Too many tools degrade selection, and agents use tools outside their specialisation. Scoping tools to the role fixes both. B is advisory. A adds tokens without removing the temptation. C also removes any legitimate tools the synthesis agent needs.

Question 23 · Domain 2: Tool Design & MCP Integration

Correct answer: C

Forcing a specific tool guarantees that call happens first. A guarantees some tool call but not which one. B relies on an unsupported assumption about ordering. D is probabilistic.

Question 24 · Domain 2: Tool Design & MCP Integration

Correct answer: B

"any" guarantees a structured tool call while keeping model-driven selection. C still allows text responses. D mis-extracts receipts and contracts. A creates a bloated schema that invites empty or fabricated fields.

Question 25 · Domain 2: Tool Design & MCP Integration

Correct answer: D

Project-scoped config shares the server definition, and environment variable expansion keeps secrets out of version control. A leaks a credential. C works but is not shared configuration, and it drifts. B puts a secret into model context.

Question 26 · Domain 2: Tool Design & MCP Integration

Correct answer: C

Purpose-specific tools with clear contracts make selection and outputs predictable. B is an improvement but keeps overloaded behaviour behind one name. D wastes tokens and buries the needed output. A adds retries without fixing the confusing interface.

Question 27 · Domain 2: Tool Design & MCP Integration

Correct answer: B

MCP resources expose content catalogues and data to the client directly, which gives the agent visibility without trial-and-error calls. C floods the context. A accepts the waste. D is manual and goes stale.

Domain 3: Claude Code Configuration & Workflows (20%)

Covers: CLAUDE.md hierarchy and @import; .claude/rules/ path-scoped rules; /memory; custom slash commands; skills (context: fork, allowed-tools, argument-hint); plan mode vs direct execution; Explore subagent; CI flags; iterative refinement patterns.

Question 28. A new team member's Claude Code sessions ignore the team's coding conventions. The conventions work for the tech lead, who wrote them in ~/.claude/CLAUDE.md. What is the cause and fix?

- **A.** CLAUDE.md files only load after the /init command has been run. The new member should run /init in the repository.
- **B.** The conventions are too long. Shorten them to under 200 lines so that Claude Code loads them reliably for every user.
- **C.** User-level memory is not shared through version control. Move the conventions into the project's CLAUDE.md in the repository.
- **D.** The new member's Claude Code version is outdated. Upgrading to the latest version will load the lead's conventions automatically.

Question 29. Test files are spread across many directories (for example, src/auth/login.test.tsx and src/cart/cart.test.tsx). The team wants testing conventions applied whenever Claude edits a test file, wherever it lives. What is the best mechanism?

- **A.** A file in .claude/rules/ with YAML frontmatter whose paths glob (for example, **/*.test.tsx) scopes the rules to test files.
- **B.** Add the testing conventions to the root CLAUDE.md, under a heading that says they apply only to test files.
- **C.** A directory-level CLAUDE.md in each folder that contains tests, each holding a copy of the same testing conventions.
- **D.** A custom slash command, /test-conventions, that developers run before asking Claude to edit any test file.

Question 30. A monorepo's root CLAUDE.md has grown to 1,200 lines covering API, frontend, data, and infrastructure standards, each maintained by a different team. Editing it causes merge conflicts, and the content is hard to navigate. What is the best restructuring?

- **A.** Move each team's section into that team's members' user-level CLAUDE.md files, so the standards no longer conflict.
- **B.** Move the entire contents into a single skill, so the standards load only when a developer explicitly invokes that skill.
- **C.** Compress the file by removing examples and rationale, keeping only the rules themselves to cut it to about 400 lines.
- **D.** Split the standards into topic-specific files owned by each team, and reference them from CLAUDE.md with @import.

Question 31. A developer reports that Claude Code follows the team's conventions in some sessions but not others, and suspects some memory files are not loading. What is the most direct diagnostic step?

- **A.** Reinstall Claude Code, clear its cache directory, and start a new session in the repository's root folder.
- **B.** Run the /memory command to see which memory files are loaded in the current session.
- **C.** Delete and recreate every CLAUDE.md file in the repository to clear any corruption in the existing files.
- **D.** Add a line to CLAUDE.md asking Claude to confirm at the start of each session that it has read the conventions.

Question 32. The team wants a /review slash command that runs its standard review checklist and is available to everyone who clones the repository. Where should it be defined?

- **A.** As a Markdown file in the repository's .claude/commands/ directory, committed to version control.
- **B.** As a Markdown file in each developer's ~/.claude/commands/ directory, distributed through the team wiki.
- **C.** As an MCP prompt exposed by a dedicated internal MCP server that every developer configures locally.
- **D.** As a section in the root CLAUDE.md that describes the checklist and tells Claude to apply it when asked to review.

Question 33. A codebase-analysis skill produces long, verbose exploration output. After it runs, the main conversation is cluttered, and later answers lose focus. What SKILL.md frontmatter setting addresses this?

- **A.** A lower max_tokens value in the frontmatter, so the skill's output is truncated before it reaches the main context.
- **B.** allowed-tools limited to Read and Grep, so the skill produces less output by being unable to use any other tools.
- **C.** argument-hint, so developers pass a narrower scope when invoking the skill and it explores fewer files overall.
- **D.** context: fork, so the skill runs in an isolated subagent context and returns only its result to the main conversation.

Question 34. A team's database-inspection skill should only ever read data. Developers worry it might write files or run shell commands while it is active. Which SKILL.md mechanism addresses this?

- **A.** The allowed-tools frontmatter field, restricting the skill to read-only tools while it is running.
- **B.** The context: fork frontmatter field, so that any writes happen in an isolated context and are discarded afterwards.
- **C.** The argument-hint field, describing which tables may be read, so Claude stays within the intended scope of the skill.
- **D.** A sentence in the skill's instructions telling Claude to never write files or execute commands while using the skill.

Question 35. A developer must restructure a monolith into microservices, which affects about 45 files. There are several valid ways to draw the service boundaries. How should they use Claude Code?

- **A.** Use direct execution file by file, with the developer deciding the service boundaries ad hoc as each file is reached.
- **B.** Use direct execution with frequent commits, so that any problems can be rolled back as the restructuring proceeds.
- **C.** Ask Claude to make all the changes in one pass, then review the full diff once at the end of the work.
- **D.** Start in plan mode to explore the codebase and design the approach, then implement the plan once it has been reviewed.

Question 36. A CI pipeline needs Claude Code's review findings as machine-parseable output that conforms to a defined structure, so a script can post them as inline PR comments. Which invocation is most appropriate?

- **A.** claude -p with a prompt that asks for JSON, then a regular expression that extracts the JSON block from the text output.
- **B.** Run Claude Code interactively in the CI container, and capture its terminal output with a screen-scraping utility.
- **C.** claude -p with default text output, plus a second Claude call that converts the text findings into JSON for the script.
- **D.** claude -p with --output-format json and --json-schema, so the output is JSON conforming to the required structure.

Question 37. During a long debugging task, Claude Code's discovery phase (reading logs, grepping, listing directories) fills the context with verbose output, and later reasoning degrades. What is the recommended approach?

A. Turn off the logs and grep output by restricting Claude Code to the Read tool, so that it produces less verbose discovery output.

B. Paste all the relevant logs into the first message up front, so Claude does not need to run any discovery commands itself.

C. Start a fresh session after every 20 tool calls, without carrying over any summary from the previous session.

D. Delegate the discovery to the Explore subagent, which works in its own context and returns a concise summary to the main session.

Question 38. Asked to "normalise the address fields", Claude Code produces a different interpretation each time, and prose clarifications have not helped. What is the most effective next step?

A. Ask Claude to choose its own interpretation and document it in a comment at the top of the transformation code.

B. Provide two or three concrete input-to-output examples that show exactly how addresses should be transformed.

C. Switch to plan mode and ask Claude to propose several possible interpretations of what normalising addresses means.

D. Rewrite the instruction as a longer paragraph covering every rule in more precise prose, including the edge cases.

Question 39. A developer is implementing a caching layer in a domain they know little about. They want to avoid missing important design considerations. Which Claude Code workflow fits best?

A. Ask Claude to implement the most common caching pattern immediately, and then iterate on whatever problems appear in production.

B. Ask Claude to interview them first, asking questions about invalidation, failure modes, and constraints before implementing.

C. Write a detailed specification alone first, and give it to Claude only once every design decision has been fully settled.

D. Ask Claude to generate three different implementations at once, then pick whichever one looks the cleanest to the developer.

Answers & Explanations: Domain 3

Questions 28–39

Question 28 · Domain 3: Claude Code Configuration & Workflows

Correct answer: C

~/.claude/CLAUDE.md applies only to that user's machine. Team conventions belong in project-level memory committed to the repository. A, B, and D misdiagnose a scope problem.

Question 29 · Domain 3: Claude Code Configuration & Workflows

Correct answer: A

Path-scoped rules apply conventions by file pattern across the codebase, and load only when relevant. C duplicates content and misses new directories. B loads the rules everywhere and relies on inference. D depends on developers remembering to run it.

Question 30 · Domain 3: Claude Code Configuration & Workflows

Correct answer: D

@import keeps memory modular, with clear ownership, while the content still loads as project memory. B makes always-needed standards on-demand. A stops sharing them. C removes context that helps Claude apply the rules correctly.

Question 31 · Domain 3: Claude Code Configuration & Workflows

Correct answer: B

/memory shows directly which memory files are loaded, which turns guessing into evidence. A and C are destructive guesses. D relies on the model's self-report.

Question 32 · Domain 3: Claude Code Configuration & Workflows

Correct answer: A

Project-scoped commands in .claude/commands/ are shared automatically through the repository. B is personal and drifts. D is not a slash command. C adds infrastructure for something a committed file already provides.

Question 33 · Domain 3: Claude Code Configuration & Workflows

Correct answer: D

Forked context isolates the verbose work and returns only the result. B restricts capability, not context pollution. C may narrow scope but still pollutes the context. A is not a skill setting and would truncate useful output.

Question 34 · Domain 3: Claude Code Configuration & Workflows

Correct answer: A

allowed-tools limits which tools the skill can use, which enforces read-only behaviour. B isolates context, not side effects, and writes would still happen. D is advisory. C only documents the expected parameters.

Question 35 · Domain 3: Claude Code Configuration & Workflows

Correct answer: D

Large, architectural changes with several valid approaches benefit from plan mode, which allows investigation and design before any edits are made. B and C commit to an approach without exploring alternatives. A creates inconsistent boundaries.

Question 36 · Domain 3: Claude Code Configuration & Workflows

Correct answer: D

These CLI flags produce structured, schema-conforming output for automation. A is brittle parsing. B cannot work in a non-interactive pipeline. C adds cost and another failure point.

Question 37 · Domain 3: Claude Code Configuration & Workflows

Correct answer: D

Isolating verbose exploration in a subagent preserves the main context for reasoning. A cripples investigation. B front-loads even more raw text. C throws away progress.

Question 38 · Domain 3: Claude Code Configuration & Workflows

Correct answer: B

When prose is interpreted inconsistently, concrete input/output examples remove the ambiguity. D repeats the approach that failed. A accepts arbitrary behaviour. C lists options but does not specify the transformation you want.

Question 39 · Domain 3: Claude Code Configuration & Workflows

Correct answer: B

The interview pattern surfaces considerations the developer may not know to specify. A finds design gaps late and expensively. C is limited by the developer's own unfamiliarity. D compares code without understanding the requirements.

Domain 4: Prompt Engineering & Structured Output (20%)

Covers: Explicit criteria vs vague instructions; false-positive management; few-shot examples; tool use with JSON schemas; nullable fields and enums; validation-retry loops; Message Batches API; independent and multi-pass review.

Question 40. An automated code reviewer flags many issues developers consider noise. Adding "Be conservative and only report high-confidence issues" made no measurable difference. What is the better approach?

- **A.** Use stronger wording such as "ONLY report issues you are absolutely certain about", and repeat it at the end of the review prompt.
- **B.** Specify explicit categorical criteria for what to report (for example, bugs and security issues) and what to skip (minor style, local conventions).
- **C.** Lower the temperature to 0, so the reviewer behaves more deterministically and produces fewer speculative findings.
- **D.** Ask the reviewer to give a confidence score from 1 to 10 with each finding, and post only the findings that score 8 or higher.

Question 41. An automated reviewer's "naming conventions" category has a 60% false-positive rate. Developers have started ignoring all its comments, including accurate security findings. What should the team do first?

- **A.** Increase the volume of security findings by lowering their threshold, so developers notice the more important issues.
- **B.** Keep all the categories enabled but add a disclaimer to each comment saying it may be inaccurate and should be checked.
- **C.** Temporarily disable the high false-positive category to restore trust, while improving its prompt criteria separately.
- **D.** Raise the severity labels of the security findings so that they stand out more clearly among the naming comments.

Question 42. An agent handles ambiguous cases inconsistently. For example, it is unclear whether a partially used gift card should be refunded or replaced. Detailed prose rules have not helped. What is the most effective technique?

- **A.** Add two to four targeted few-shot examples of ambiguous cases, showing the reasoning for choosing one action over the plausible alternatives.
- **B.** Add a dozen more prose rules covering each specific gift card scenario that has been observed in the support logs to date.
- **C.** Ask the agent to always choose the more conservative action whenever a case seems ambiguous to it, without explaining the reasoning.
- **D.** Route every ambiguous case to a human, since ambiguity means the policy cannot be applied consistently by the agent.

Question 43. An extraction pipeline asks Claude to "return JSON matching this schema" in the prompt. About 3% of responses have syntax errors such as trailing commas or unescaped quotes. What is the most reliable fix?

- **A.** Add a JSON repair library that automatically fixes trailing commas and quotes after each response is received.
- **B.** Define the schema as a tool's input schema and have Claude return data through tool use, which removes JSON syntax errors.
- **C.** Add three examples of perfectly formatted JSON responses to the prompt, so that Claude copies the correct syntax.
- **D.** Retry every failed parse up to three times, appending the parser's error message to the original prompt.

Question 44. Invoice extraction now uses tool use with a strict schema. No more syntax errors occur, but in some outputs the line items do not add up to the stated total. What is the best improvement?

A. Add a schema constraint requiring the line items to sum to the total, so that invalid outputs are rejected by the schema.

B. Ask Claude in the prompt to double-check its arithmetic before returning the extracted invoice data.

C. Extract a calculated_total alongside the stated_total and flag discrepancies, since schemas cannot catch semantic errors.

D. Switch to a larger model, which will make fewer extraction mistakes on line items and totals.

Question 45. An extraction schema marks every field as required. For documents that lack some information (for example, no PO number), Claude fills those fields with plausible-looking invented values. What schema change best prevents this?

A. Make fields that may be absent optional or nullable, so Claude can return null instead of inventing a value.

B. Use default placeholder strings such as "N/A" in the schema, and require Claude to use them for missing fields.

C. Require a confidence score for every field, and discard any extraction where a confidence score falls below 0.9.

D. Add a prompt instruction never to invent values, while keeping all fields required in the schema.

Question 46. A ticket-classification schema has a required category enum with six values. About 15% of tickets do not fit any of them, and Claude forces them into the closest category, polluting the reports. What is the best schema fix?

A. Add a prompt instruction to pick the closest category, but to mention any uncertainty in the ticket's summary field.

B. Remove the enum constraint entirely and let Claude write any category name it thinks fits each ticket.

C. Add twenty more specific categories to the enum, so that every observed ticket type has an exact match in the list.

D. Add an "other" enum value paired with a free-text detail field, so tickets outside the categories are captured honestly.

Question 47. An extraction fails validation: the date field is "03/04" (ambiguous) and the currency is missing. What is the most effective retry strategy?

A. Send a follow-up request with the original document, the failed extraction, and the specific validation errors to correct.

B. Resend the request with a higher temperature, so that the model explores different interpretations of the date and currency.

C. Resend the original request unchanged, because extractions are non-deterministic and a second attempt often succeeds.

D. Send only the validation errors without the document, and ask Claude to output the corrected values it should have produced.

Question 48. A validation-retry loop for contract extraction resolves most errors, but one class never resolves: the governing-law clause is "as specified in the Master Agreement", which is a separate document. What does this indicate?

A. The retry prompt is not specific enough. Adding more detailed error feedback will eventually produce the governing-law value.

B. The schema is wrong. The governing-law field should be removed, since it cannot be filled reliably from contracts.

C. Retries cannot help when the information is absent from the source. These cases need the referenced document or a human, not more retries.

D. The model lacks legal knowledge. A larger model will be able to infer the likely governing law from the rest of the contract.

Question 49. A manager proposes moving two workflows to the Message Batches API for its 50% cost saving: (1) a blocking pre-merge check that developers wait on, and (2) a nightly technical-debt report. What is the right recommendation?

A. Move the pre-merge check to batch processing and keep the nightly report synchronous, since the nightly report runs on a timer.

B. Use batch processing only for the nightly report. Batches can take up to 24 hours with no latency SLA, which does not suit blocking checks.

C. Move both workflows to batch processing, since most batches complete within minutes and the cost savings are significant.

D. Keep both workflows on synchronous calls, since batch processing does not support the tool definitions that both workflows rely on.

Question 50. In a CI pipeline, the same Claude Code session that generated a feature's code is then asked to review it. It rarely finds issues that human reviewers later catch. What is the best change?

A. Add extended thinking to the generating session's self-review, so it reasons more carefully about potential defects.

B. Run the review in an independent Claude instance that does not have the generation session's reasoning context.

C. Ask the generating session to review its own code twice, with an instruction to be especially critical the second time.

D. Have the generating session list its assumptions first, and then review the code against those assumptions.

Question 51. A single review pass over a 14-file pull request gives detailed feedback on some files, superficial comments on others, and misses a data-flow bug that spans two files. What review architecture is recommended?

A. Use a larger model with a bigger context window, so that all 14 files receive equal attention in one review pass.

B. Run per-file passes for local issues, plus a separate integration pass focused on cross-file data flow.

C. Ask developers to split large pull requests so that no PR ever contains more than five changed files.

D. Run the same single-pass review three times and keep only the issues flagged in at least two of the three runs.

Answers & Explanations: Domain 4

Questions 40–51

Question 40 · Domain 4: Prompt Engineering & Structured Output

Correct answer: B

Vague instructions such as "be conservative" do not define the boundary, and explicit criteria do. D relies on poorly calibrated self-assessment. A adds emphasis without information. C changes randomness, not the review criteria.

Question 41 · Domain 4: Prompt Engineering & Structured Output

Correct answer: C

False positives in one category erode trust in all findings. Removing the noisy category protects the valuable ones while it is fixed. B and D leave the noise in place. A adds more noise.

Question 42 · Domain 4: Prompt Engineering & Structured Output

Correct answer: A

Few-shot examples that demonstrate the reasoning behind a judgement call generalise better than more rules. B has already failed. C is vague and still inconsistent. D over-escalates cases the agent could handle.

Question 43 · Domain 4: Prompt Engineering & Structured Output

Correct answer: B

Tool use with a JSON schema yields structurally valid output. C reduces but does not remove errors. A and D patch the symptoms after the fact.

Question 44 · Domain 4: Prompt Engineering & Structured Output

Correct answer: C

Schemas enforce structure, not cross-field arithmetic. A self-check field makes discrepancies visible and actionable. A is not expressible in a standard JSON schema. B and D may help but give no detection.

Question 45 · Domain 4: Prompt Engineering & Structured Output

Correct answer: A

Required fields pressure the model to produce something, and nullable fields give it an honest option. D conflicts with the schema's demand. C relies on calibrated self-assessment. B complicates downstream validation compared with a real null.

Question 46 · Domain 4: Prompt Engineering & Structured Output

Correct answer: D

An extensible "other" value with detail preserves data quality and shows new categories. C adds complexity without guaranteeing coverage. B loses consistency. A still forces wrong categories.

Question 47 · Domain 4: Prompt Engineering & Structured Output

Correct answer: A

Error-specific feedback, together with the source and the previous attempt, gives the model what it needs to correct itself. B and C ignore the known problem. D removes the source needed to resolve it.

Question 48 · Domain 4: Prompt Engineering & Structured Output

Correct answer: C

Retries fix format and structure errors, not missing information. A wastes calls. D invites fabrication. B discards a field that is valuable when the information is present.

Question 49 · Domain 4: Prompt Engineering & Structured Output

Correct answer: B

Batch processing fits latency-tolerant work, not anything a developer is waiting on. C has no latency guarantee. D is wrong, because batch requests can include tool definitions. A is backwards.

Question 50 · Domain 4: Prompt Engineering & Structured Output

Correct answer: B

A session that carries its own generation reasoning tends to confirm its earlier decisions. An independent instance reviews with fresh eyes. A, C, and D keep the same biased context.

Question 51 · Domain 4: Prompt Engineering & Structured Output

Correct answer: B

Splitting the review avoids attention dilution, and a dedicated integration pass catches cross-file issues. A does not solve uneven attention. C shifts the burden onto process. D discards real bugs that are caught only some of the time.

Domain 5: Context Management & Reliability (15%)

Covers: Case-facts preservation; lost-in-the-middle; trimming tool output; error propagation in multi-agent systems; conflicting sources and provenance; long-session degradation; crash recovery; human review sampling and calibration.

Question 52. In long support conversations, older turns are progressively summarised to save context. Agents then quote wrong refund amounts and order numbers that the customer gave early on. What is the best fix?

- **A.** Stop summarising and keep the full conversation history, even when it exceeds the context budget for long conversations.
- **B.** Keep a persistent "case facts" block with exact amounts, dates, and IDs, included in each prompt outside the summarised history.
- **C.** Have the agent ask the customer to confirm amounts and order numbers again before taking any action on the case.
- **D.** Ask the summariser to be more careful about numbers, and to preserve all monetary values in each summary it produces.

Question 53. A coordinator passes Claude a long aggregated input of 12 subagent reports. Findings that appear in the middle of the input are often omitted from the synthesis. Which input organisation best addresses this?

- **A.** Shuffle the order of the reports for each request, so that no particular report is consistently in the middle.
- **B.** Put a key-findings summary at the beginning, and organise the detailed results under explicit section headers.
- **C.** Concatenate the reports without headers, so the model reads them as one continuous narrative rather than separate pieces.
- **D.** Put the most important report last, since content at the end of the input always receives the most attention.

Question 54. Each order lookup returns over 40 fields, but only 5 matter for return-related questions. In multi-step cases the context fills with irrelevant fields and quality drops. What is the best fix?

- **A.** Ask the agent to ignore the irrelevant fields when reading lookup results during its reasoning process.
- **B.** Increase the context window budget, so that long multi-step cases have room for every field from every lookup.
- **C.** Limit each case to two order lookups, so the context never accumulates more than about 80 fields in total.
- **D.** Trim tool outputs to the relevant fields before they accumulate in context (for example, in the tool or a post-processing hook).

Question 55. A web-search subagent times out on one of its queries. Which response to the coordinator best supports good recovery decisions?

- **A.** A structured error with the failure type, the query attempted, any partial results, and possible alternatives such as other sources.
- **B.** A generic "search unavailable" message, so the coordinator knows that this subagent's part of the research has failed.
- **C.** An exception that terminates the entire research workflow, so no report is produced from incomplete research.
- **D.** An empty result marked as successful, so the overall workflow continues without interruption from the single failure.

Question 56. Two credible sources report different market-size figures for the same segment: $4.2B and $5.1B. How should the synthesis agent handle this?

A. Report both figures with their source attribution and note the conflict, including any differences in methodology or date.

B. Choose the figure from the more recent source, and omit the other to keep the report clear and decisive for readers.

C. Average the two figures to $4.65B, and cite both sources as supporting the averaged market-size estimate in the report.

D. Exclude market size from the report entirely, since conflicting data makes the figure unreliable for decision-making.

Question 57. During a long codebase exploration, Claude Code starts referring to "typical patterns" instead of the specific classes and functions it found earlier. Which practices best counter this degradation?

A. Ask Claude to try harder to remember specifics, by adding "be precise about class names" to the project's CLAUDE.md.

B. Restart the session from scratch every hour without carrying anything forward, to keep the context window clean.

C. Persist key findings to a scratchpad file the agent can re-read, delegate focused questions to subagents, and use /compact when needed.

D. Increase the session's output token limit, so that Claude has room to restate all its earlier findings in each response it gives.

Question 58. A multi-hour, multi-agent analysis job crashed halfway through. Restarting from the beginning would repeat four hours of work. Which design enables efficient recovery?

A. Run the whole job inside one long session with --resume, so that any crash can be resumed from the last message.

B. Keep everything in the coordinator's conversation history, so a resumed session automatically includes all the completed work.

C. Have each agent export its state to a known location (a manifest), and have the coordinator load it on resume.

D. Rerun the job from the start but with a faster model, so repeating the completed work takes less time.

Question 59. An invoice-extraction system reaches 97% overall accuracy. Leadership wants to remove human review for all high-confidence extractions. What should be done first?

A. Keep human review for all documents permanently, since any automation error in finance is unacceptable.

B. Lower the confidence threshold so that more extractions qualify as high-confidence, and then remove human review for them.

C. Remove human review immediately, since 97% accuracy exceeds the typical human data-entry accuracy benchmark.

D. Analyse accuracy by document type and field. Aggregate accuracy can hide categories with much worse performance.

Question 60. In a research system, the final report contains claims, but reviewers cannot tell which source supports which claim, because attribution was lost during synthesis. What design prevents this?

A. Have a separate agent search the web after synthesis to find sources that support each claim in the report.

B. Add a bibliography at the end of the report that lists every source the subagents consulted during the research.

C. Require subagents to output structured claim-source mappings (URL, document, excerpt, date) that synthesis must preserve.

D. Instruct the synthesis agent to remember where each claim came from and cite sources wherever it can.

Answers & Explanations: Domain 5

Questions 52–60

Question 52 · Domain 5: Context Management & Reliability

Correct answer: B

Summarisation loses specifics. Extracting critical transactional facts into a durable block preserves them exactly. D remains lossy. A breaks context limits. C burdens the customer.

Question 53 · Domain 5: Context Management & Reliability

Correct answer: B

Models can under-weight content in the middle of long inputs. Up-front summaries and clear structure counter this. A randomises which findings get missed. D helps only one report. C removes structure that aids attention.

Question 54 · Domain 5: Context Management & Reliability

Correct answer: D

Removing irrelevant data before it enters context keeps the agent focused and saves tokens. B delays the problem. A still spends attention on the noise. C blocks legitimate multi-order cases.

Question 55 · Domain 5: Context Management & Reliability

Correct answer: A

Rich error context lets the coordinator retry, re-route, or proceed with an explicit caveat. B hides what failed and why. D silently passes a gap off as a valid finding. C discards all the work that succeeded.

Question 56 · Domain 5: Context Management & Reliability

Correct answer: A

Preserving and annotating the conflict with provenance lets readers judge it. C invents a number neither source supports. B arbitrarily hides evidence. D removes useful information.

Question 57 · Domain 5: Context Management & Reliability

Correct answer: C

Externalised notes, isolated subagent work, and compaction keep important specifics available as context grows. D does not affect input context. B throws away progress. A is advisory.

Question 58 · Domain 5: Context Management & Reliability

Correct answer: C

Durable, structured state outside any one context window supports resumption after failures. A and B rely on conversation state that may be incomplete or stale after a crash. D still repeats the work.

Question 59 · Domain 5: Context Management & Reliability

Correct answer: D

Segment-level analysis, such as by document type and field, shows where automation is safe. C ignores possible weak segments. A gives up value without evidence. B increases risk.

Question 60 · Domain 5: Context Management & Reliability

Correct answer: C

Provenance must be carried as structured data through every hand-off. B gives no claim-level linkage. D is unreliable once the mapping has been lost. A back-fills citations that may not be the real basis for the claims.

Answer Key & Domain Score Sheet

Q	Ans	Dom	Q	Ans	Dom	Q	Ans	Dom	Q	Ans	Dom
1	C	D1	16	B	D1	31	B	D3	46	D	D4
2	D	D1	17	C	D2	32	A	D3	47	A	D4
3	A	D1	18	D	D2	33	D	D3	48	C	D4
4	A	D1	19	A	D2	34	A	D3	49	B	D4
5	D	D1	20	D	D2	35	D	D3	50	B	D4
6	C	D1	21	C	D2	36	D	D3	51	B	D4
7	D	D1	22	D	D2	37	D	D3	52	B	D5
8	B	D1	23	C	D2	38	B	D3	53	B	D5
9	A	D1	24	B	D2	39	B	D3	54	D	D5
10	A	D1	25	D	D2	40	B	D4	55	A	D5
11	B	D1	26	C	D2	41	C	D4	56	A	D5
12	A	D1	27	B	D2	42	A	D4	57	C	D5
13	A	D1	28	C	D3	43	B	D4	58	C	D5
14	C	D1	29	A	D3	44	C	D4	59	D	D5
15	C	D1	30	D	D3	45	A	D4	60	C	D5

Domain	Weight	Your score	Question numbers
D1: Agentic Architecture & Orchestration	27%	/ 16	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16
D2: Tool Design & MCP Integration	18%	/ 11	17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27
D3: Claude Code Configuration & Workflows	20%	/ 12	28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39
D4: Prompt Engineering & Structured Output	20%	/ 12	40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51
D5: Context Management & Reliability	15%	/ 9	52, 53, 54, 55, 56, 57, 58, 59, 60
Total	100%	/ 60	

Reading your result: 48+ overall with no domain below 70% suggests you are ready to book. For any weaker domain, re-read its explanations here, then test yourself on the same domain in Set 2 (the scenario mock exam).